DATABASE MANAGEMENT SYSTEMS · WEEK 01 + LAB 01

Welcome to the Data Universe: Files vs DBMS

Why a flat file stops working when KoboKart grows, and what a database gives us instead.

Dr.-Ing. Chukwuka Victor Obionwu

TODAY'S JOURNEY

Two hours, six stops

Time	Stop	The question we answer
0:00	Warm-up	How would you store a shop's orders?
0:10	Flat files	What breaks when KoboKart gets busy?
0:32	What a DBMS gives us	Rules, speed, safety: how?
0:55	PostgreSQL	Why this tool, and why DECIMAL for money?
1:02	Docker preview	How do we run it on a laptop? (break at 1:10)
1:15	Lab 01 (after the break)	Is SQL really faster than a file?
1:50	Exit quiz	What stuck?

Every stop has a short exercise. Answers appear on the next click.

WHERE WE ARE GOING

By the end of today you can…

1. Name three ways a flat file fails when many users and lots of data arrive.
2. Say what a database management system (DBMS) adds: rules, indexes, SQL, transactions and crash recovery.
3. Read a CREATE TABLE statement and explain each constraint.
4. Explain why an index makes a lookup fast, using a real measurement.
5. Start PostgreSQL 16 in Docker, load 100,000 orders and benchmark file against SQL.

No database experience is assumed. If a word is new, it gets a NEW WORD box.

HOW WE WORK TODAY

Four signs to watch for

YOUR TURN
Stop and try it on paper or on your laptop. A timer is shown.

NEW WORD
A technical term, explained in plain words before we use it.

QUICK CHECK
A multiple-choice question. Vote with your fingers: 1 to 4.

STUCK?
Raise a hand or write the exact message on the board. Mistakes are the point.

Every answer is hidden until you have committed to one. Numbers you see today were measured on a real machine.

WARM-UP · YOUR TURN

How would you store a shop's orders?

KoboKart is a new online marketplace where many merchants sell to shoppers across West Africa. You only have a plain text file and a programming language. Nothing else.

YOUR TURN (2 minutes)
On paper: (1) sketch one line of the file for one order, using columns; (2) list **three things that could go wrong** once thousands of shoppers order at the same time.

The file: something like order_id, customer_id, merchant_id, total, status, one line per order.

Most lists contain: two shoppers saving at once, the file getting slow, and wrong values slipping in. Those are exactly today's three villains. First we see them fail, then we meet the tool built to stop them.

MEET KOBOKART

A flash sale is coming

In the first version of KoboKart, every order is one line in **orders.csv**. It works well for the first hundred orders. Then marketing announces a flash sale.

VILLAIN 1

Lost writes

Many programs write the same file at once.

VILLAIN 2

Slow search

Finding one customer means reading every line.

VILLAIN 3

No rules

Nothing blocks a negative total.

The flash sale: up to 10,000 checkout attempts per second. We will test each villain on a real file with 100,000 orders.

WHAT IS IN THE FILE?

A file is just text

```
order_id,customer_id,merchant_id,total,status
1001,usr_881,mer_001,450.00,PAID
1002,usr_902,mer_002,1200.50,PENDING
1003,usr_881,mer_001,89.99,SHIPPED
```

The computer sees only characters and commas. It does not know that “total” must be a number, or that “status” should be one of a few words.

YOUR TURN (60 seconds)

Write the steps a program must follow to find **all orders of usr_881**.

Open the file, read line 1, split it at the commas, compare the customer column, then line 2, line 3… until the very last line. There is no shortcut.

QUICK CHECK 1 · VOTE WITH FINGERS

Which statement about orders.csv is true?

A. The file knows that total must be a number.

B. Any program can write any text into any line.

C. The file blocks two programs from writing at once.

D. The file keeps itself sorted for fast search.

B. A file has no rules and no guard. Whatever a program writes, the file keeps.

A, C and D describe things a database does, not a text file.

VILLAIN 1 · PREDICT FIRST

Two programs, one file

Two checkout programs each save **300 new orders**. Each one does the same three steps for every order:

1. Read the whole file → **2. Add one line** → **3. Write the file back**

YOUR TURN: vote with fingers

How many orders will the file hold at the end? 1 = 600 · 2 = about 450 · 3 = about 300 · 4 = far fewer

Predictions first, then click. Think about what happens when both programs read the file at the same moment.

VILLAIN 1 · MEASURED RESULT

Most orders vanished

REAL RUNS · 2 programs × 300 orders = 600 expected

Run 1: **64** orders survived
Run 2: **53** orders survived

NEW WORD · Race condition: the result depends on which program happens to be first.

Step	Program A	Program B
1	reads 10 lines	reads 10 lines
2	writes back 11	
3		writes back 11
End	file has 11 lines	B's copy won

A's new line is gone. B started from the old copy and wrote over it. With thousands of writes per second, almost every order is lost, and nobody gets an error message.

QUICK CHECK 2 · VOTE WITH FINGERS

What do we call this bug?

A. A race condition

B. A syntax error

C. An index miss

D. A rollback

A. Two programs race, and the winner decides what the file contains. A database prevents this with locks and transactions, which we meet soon.

VILLAIN 2 · SLOW SEARCH

Reading everything, every time

Imagine a phone book with names in random order. To find one person you must read every entry. A file of orders works the same way.

NEW WORD · Linear search: check items one by one from the start. If the file has N lines, the work grows in step with N. Computer scientists write this as **O(N)**.

YOUR TURN (45 seconds)

If one lookup takes 1 second for 100,000 lines, how long for 1,000,000 lines?

About 10 seconds. Ten times more lines means ten times more reading. Doubling the data doubles the wait.

VILLAIN 2 · MEASURED

What one lookup really costs

Task on 100,000 orders in a CSV file	Measured	Could serve per second
All orders of one customer	0.24 s	about 4
Group pending orders by customer	0.22 s	about 4

Flash sale: 10,000 requests per second are arriving, one program can answer about 4. Even many programs together would not keep up.

Times were measured with Python's csv module on one CPU core. If the file grows ten times, expect roughly ten times longer (about 2.4 s per lookup at 1,000,000 lines).

The slowness is not a bug in our code. It is the shape of the problem: a file gives no way to jump straight to the right line.

QUICK CHECK 3 · VOTE WITH FINGERS

How does O(N) behave?

A lookup in a file takes 3 seconds. The file now doubles in size. What is the new lookup time?

A. Still about 3 seconds

B. About 6 seconds

C. About 9 seconds

D. About 1.5 seconds

B. Linear search: double the lines, double the work.

VILLAIN 3 · YOUR TURN

Spot the bad lines

The mobile app has a bug. Merchants known to KoboKart so far: **mer_001** and **mer_002**. Here are the new lines it wrote:

```
1004,usr_881,mer_999,450.00,PAID
1005,usr_902,mer_001,-450.00,PAID
1006,,mer_002,abc,SHIPPED
```

YOUR TURN (60 seconds)
Circle everything that should not be allowed.

1004: merchant mer_999 does not exist. **1005:** a negative total. **1006:** no customer, and “abc” is not a number.

The file accepted all three lines without a single complaint.

QUICK CHECK 4 · TRUE OR FALSE

Three statements, one finger each

1. A CSV file can refuse to save a negative total.

2. Every program that reads the file must check the data itself, because it cannot trust the lines.

3. If one forgetful program skips the checks, bad data can enter the file.

1. False. A file has no rules. **2. True. 3. True.** Rules that live in many programs are rules that will eventually be forgotten. We want the rules to live with the data.

RECAP

Three villains, three measurements

Villain	What we saw	What we need instead
Lost writes	53 to 64 of 600 orders kept	Safe sharing between programs
Slow search	0.24 s per lookup, 100,000 lines	A way to jump to the right rows
No rules	Negative totals and missing merchants accepted	Rules enforced by the storage itself

The tool that provides all three is a **database management system**. That is next.

NEW WORD · DBMS

One guard between programs and data

Many programs
website, phone app, reports

→

DBMS
the guard that checks, orders and protects

→

Data on disk
stored safely, never touched directly

DBMS (database management system): specialized software that stores data and manages it **efficiently, safely and concurrently**. "Concurrently" means many users at the same time.

Rules
bad data is refused

Speed
indexes jump to rows

Safe sharing
transactions

Recovery
a log survives crashes

VILLAIN 3 DEFEATED · READ THE CODE

Rules written into the table

```
-- KoboKart orders table in PostgreSQL 16
CREATE TABLE orders (
  order_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id UUID NOT NULL,
  merchant_id UUID NOT NULL,
  order_total DECIMAL(10, 2) NOT NULL CHECK (order_total ⩾ 0.00),
  status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

PRIMARY KEY: each row is unique and never empty.

NOT NULL: this value is required.

CHECK: a rule every row must satisfy.

DEFAULT: the value used when none is given.

DECIMAL(10, 2): up to 10 digits, 2 after the point.

UUID: a long unique id such as 797ea2ee-4cb1-…

QUICK CHECK 5 · FILL THE BLANKS ON PAPER

Design a merchants table

Each merchant has an id (auto-generated, identifies the row), a name (up to 80 characters, required) and a rating from 0 to 5 with one decimal place.

```
CREATE TABLE merchants (
  merchant_id UUID __(1)__ DEFAULT gen_random_uuid(),
  name VARCHAR(80) __(2)__,
  rating DECIMAL(2, 1) __(3)__
);
```

(1) PRIMARY KEY **(2)** NOT NULL **(3)** CHECK (rating BETWEEN 0 AND 5)

We ran this statement on PostgreSQL 16: ratings 4.5 and 5.0 are accepted, and a rating of 7.0 would be refused by the CHECK.

WATCH IT REFUSE

The database says no

```
INSERT INTO orders(customer_id, merchant_id, order_total)
VALUES (gen_random_uuid(), gen_random_uuid(), -450.00);
```

```
ERROR: new row for relation "orders"
violates check constraint "orders_order_total_check"
```

A valid order with total 450.00 and no status given is accepted, and PostgreSQL fills in **status = PENDING** by itself (the DEFAULT).

The rule lives with the data. The phone app, the website and the report script are all refused in exactly the same way. No program can forget the rule.

VILLAIN 2 DEFEATED · NEW WORD

An index is a table of contents

At the back of a textbook, the index tells you which pages mention a word, so you do not read every page. A database index does this for a column.

```
CREATE INDEX idx_orders_customer ON orders(customer_id);
```

PostgreSQL keeps customer_ids in a sorted tree (a **B-Tree**). Each step halves the search, so the work grows like **O(log N)** instead of O(N).

> **YOUR TURN (45 seconds)**
> If each step halves the search, how many steps to find one value among 1,000,000?

About 20. Doubling 20 times gives 1,048,576. Reading 1,000,000 lines versus about 20 steps is the entire point of an index.

MEASURED · ONE CUSTOMER, 100,000 ORDERS

From milliseconds to microseconds

Method	Time	How it found the rows
CSV file, Python	240 ms	read every line
PostgreSQL, no index	6.3 ms	Seq Scan: checked all rows
PostgreSQL, with index	0.07 ms	Index Scan: went to 18 rows

Same question, same 18 orders found. With the index the database skipped 99,982 other rows. That is about 90 times faster than its own scan, and thousands of times faster than the file.

Two wins: PostgreSQL stores data in a compact binary form (much faster than parsing text), and the index lets it skip almost everything.

QUICK CHECK 6 · VOTE WITH FINGERS

When does an index help most?

A. When the query needs most of the table

B. When the query picks a small slice of the table

C. When the table has only ten rows

D. When nobody ever writes to the table

B. Finding 18 rows among 100,000 is a perfect job for an index.

Remember this for the lab: its Boss Level query matches about 21% of all rows, so we will see a much smaller gain there.

NEW WORD · DECLARATIVE

Say what you want, not how

Python: you give the steps

```
for line in file:
  if line.customer == "usr_881":
    keep(line)
```

SQL: you describe the result

```
SELECT * FROM orders
WHERE customer_id = '…';
```

The query optimizer chooses the plan. We ran the identical SQL twice. Before the index PostgreSQL printed **Seq Scan**. After CREATE INDEX it printed **Bitmap Index Scan**. We never changed the query.

Lab tool: put **EXPLAIN ANALYZE** in front of any query to see the plan it chose and how long each step took.

VILLAIN 1 DEFEATED · NEW WORD · ACID

A checkout is all or nothing

A checkout has three steps: subtract the item from stock, create an invoice, charge the card. All three must succeed together.

Letter	Promise
A Atomicity	All steps succeed, or none of them count
C Consistency	Rules such as CHECK always hold after the transaction
I Isolation	Simultaneous shoppers do not corrupt each other’s work
D Durability	Once confirmed, it survives a crash

```
BEGIN; … step 1 … step 2 … step 3 …; COMMIT;  -- or ROLLBACK;
```

QUICK CHECK 7 · VOTE WITH FINGERS

The card charge failed. Now what?

Steps 1 (stock) and 2 (invoice) worked, but step 3 (charging the card) failed.

A. Keep steps 1 and 2 and hope the customer pays later

B. Roll back all three steps, as if nothing happened

C. Keep only step 3

D. Ask the customer to fix the database

B. This is atomicity. We tested it: BEGIN; INSERT an order; ROLLBACK; and the table was unchanged.

VILLAIN: THE POWER CUT

Write the plan before the change

Write-ahead logging (WAL): before PostgreSQL changes the real data, it writes a short note of the change to a log on disk. Think of a warehouse worker who writes “moving box 7 to shelf B” on a list before lifting it.

1. Note in the log → **2. Tell the user: committed** → **3. Update data files later**

YOUR TURN (30 seconds)

The power fails right after step 2. Is the confirmed order lost?

No. When the server restarts it replays the log and finishes step 3. The customer’s confirmed order survives.

RECAP

File versus DBMS

Need	Flat file	DBMS (PostgreSQL)
Many writers at once	Lost writes	Transactions, isolation
Find rows fast	Read every line	Index, about 20 steps
Block bad data	No rules	PRIMARY KEY, NOT NULL, CHECK
All or nothing	None	COMMIT or ROLLBACK
Survive a crash	Partial files	Write-ahead log

The lecture’s core takeaway: files are fine for one small script; production systems like KoboKart need a DBMS.

THE POSTGRESQL ECOSYSTEM

Why we use PostgreSQL 16

PostgreSQL is an enterprise-grade, open-source relational database: free to use, and trusted in production.

Rich data types

UUID, JSONB, NUMERIC and many more.

Several index kinds

B-Tree, Hash, GIN and GiST, each good for a different job.

Extensions

Add-ons such as pgvector, which stores AI vector data.

This course standardizes on version 16, so everyone sees the same behavior.

DATA TYPES

Each column has a type

Type	Used for	KoboKart column
UUID	Unique identifiers	order_id, customer_id
DECIMAL(10, 2)	Exact money amounts	order_total
VARCHAR(20)	Short text, 20 characters at most	status
TIMESTAMP WITH TIME ZONE	A moment in time	created_at
JSONB	Flexible, document-like data	not used in Lab 01

A type is another free guard: PostgreSQL refuses “abc” in a DECIMAL column.

DATA TYPES MATTER

Why money is never a float

YOUR TURN: predict (30 seconds)

In PostgreSQL, is 0.1 + 0.2 equal to 0.3 when stored as a float? And as DECIMAL?

Float: NOT equal. 0.1 + 0.2 = 0.30000000000000004. **DECIMAL: equal.** 0.1 + 0.2 = 0.3.

Adding 0.1 one million times: float gives **100000.00000133288**, DECIMAL gives exactly **100000.0**.

Both results were produced by running the statements on PostgreSQL 16.

QUICK CHECK 8 · VOTE WITH FINGERS

Which type for order_total?

A. FLOAT

B. DECIMAL(10, 2)

C. VARCHAR(20)

D. BOOLEAN

B. Exact amounts with two decimal places. FLOAT drifts, VARCHAR is text, and BOOLEAN is only true or false.

NEW WORD · CONTAINER

A sealed box with PostgreSQL inside

Installing a database on every laptop is messy: different systems, different versions. **Docker** lets everyone run the same sealed box instead.

Image

A ready-made template of the box and its contents.

like a recipe: postgres version 16

Container

A running copy made from an image.

like a dish cooked from the recipe: kobokart-db

Compose file

A short text file that says which containers to start.

docker-compose.yml

Why we do this: every student gets exactly PostgreSQL 16, with the same user and database names, and a mistake can be wiped by deleting the container.

READING A COMPOSE FILE

Four lines that matter

```
# Illustration only: use the file from your lab starter
services:
  postgres:
    image: postgres:16
    container_name: kobokart-db
    environment:
      POSTGRES_USER: kobokart
      POSTGRES_DB: kobokart_db
      POSTGRES_PASSWORD: <from your starter>
    ports: ["5432:5432"]
    volumes: ["./data:/data"] # CSV folder visible inside
```

image: which template to use. 16 = PostgreSQL 16.

container_name: the name you type in later commands.

environment: the database user and name to create.

ports: opens the door 5432 so tools can connect.

volumes: shares a folder from your laptop into the box.

QUICK CHECK 9 · VOTE WITH FINGERS

Image or container?

A. An image is the running copy; a container is the template

B. An image is a template; a container is a running copy of it

C. They are two names for the same thing

D. A container is a kind of SQL table

B. One image can start many containers.

In the lab, **docker compose up -d postgres** starts the container. The **-d** means “detached”: run in the background and give me my terminal back.

LAB PREVIEW

What you will build after the break

1 Start
PostgreSQL 16 in Docker

→

2 Create
raw_orders table

→

3 Load
100,000 orders

→

4 Measure
file versus SQL

Before the break: open Docker Desktop and run **docker info**. If it prints server details, you are ready. If it complains, tell me during the break and we fix it together.

Everything you saw today (race, search, rules, index) shows up in the numbers you will measure.

BREAK

Five minutes

Back at 1:15. Start Docker Desktop if it is not running yet.

LAB 01 · ENVIRONMENT BOOTSTRAP AND FILE VERSUS SQL

Three tiers, one benchmark

Warm-up · 15 min · 20 pts

Container and healthcheck

Start PostgreSQL with Docker Compose and connect with psql.

Core · 30 min · 50 pts

Table and data import

Create raw_orders and load 100,000 orders with \copy.

Boss Level · 45 min · 30 pts

Index versus flat file

Time the CSV scan, then EXPLAIN ANALYZE before and after an index.

Suggested plan for today's 35 minutes: Warm-up and Core in class, in pairs. Boss Level is homework. The tiers add up to 90 minutes, more than today's slot.

Work in pairs: one types, one reads the checkpoints. Swap after the Warm-up.

WARM-UP · YOUR TURN (15 MINUTES)

Start the database and say hello

```
# 1. start PostgreSQL in the background
docker compose up -d postgres

# 2. open a SQL shell inside the container
docker exec -it kobokart-db psql -U kobokart -d kobokart_db

# 3. at the kobokart_db=# prompt
SELECT version();
\q
```

CHECKPOINTS

- ☐ Step 1 ends without an error.
- ☐ The prompt changes to **kobokart_db=#**.
- ☐ version() says **PostgreSQL 16**.
- ☐ \q returns you to your normal terminal.

> The commands are from the lab guide. Your first start may take a minute while Docker downloads the image.

STUCK? TOTALLY NORMAL

Warm-up problems

What you see (typical message)	Likely cause	Fix
Cannot connect to the Docker daemon	Docker Desktop is not running	Start Docker Desktop, wait, run docker info
port is already allocated	Another program uses port 5432	Stop the other PostgreSQL, or ask for a different port
No such container: kobokart-db	The container did not start	docker compose ps, then docker compose up -d postgres
role "kobokart" does not exist	Wrong user, or an old data volume	Use exactly -U kobokart; ask before deleting any volume

Raise a hand or write the exact message on the board.

CORE · STEP 1 OF 2

Create the staging table

```
CREATE TABLE raw_orders (
  order_id UUID PRIMARY KEY,
  customer_id UUID NOT NULL,
  merchant_id UUID NOT NULL,
  order_total DECIMAL(10, 2) NOT NULL,
  status VARCHAR(20) NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

YOUR TURN (2 minutes)

Compare raw_orders with the orders table from the lecture. Write down **every difference**

raw_orders has no CHECK on the total and no DEFAULT on status or order_id. The ids come from the CSV, so no generator is needed.

It is a **staging** table: a landing place for raw data. We tested it: a negative total is accepted here, while a duplicate order_id is refused by the PRIMARY KEY.

CORE · STEP 2 OF 2

Load 100,000 rows with \copy

```
\copy raw_orders(order_id, customer_id, merchant_id,
        order_total, status, created_at)
FROM './data/orders_100k.csv'
WITH (FORMAT csv, HEADER true);
```

HEADER true: the first line holds column names, so skip it. **Expected reply:** COPY 100000

Then check: SELECT count(*) FROM raw_orders; should print 100000.

> **Watch the path.** \copy runs inside psql, and psql runs inside the container here. If you see “could not open file”, the CSV is not visible inside the container: check the volume line in your compose file.

CORE · PROVE IT WORKS

Let the database say no

```
INSERT INTO raw_orders
SELECT * FROM raw_orders LIMIT 1;
```

YOUR TURN: predict (30 seconds)

This copies an existing row into the same table. What will PostgreSQL do, and why?

It refuses. The row has an order_id that already exists:

```
ERROR: duplicate key value violates unique constraint "raw_orders_pkey"
```

That is the PRIMARY KEY doing its job. If you see this error after loading the CSV a second time, you loaded the data twice: run TRUNCATE raw_orders; and import once more.

BOSS LEVEL · HOMEWORK

Index versus flat file

```
import csv, time
def flat_query():
    # TODO: read orders_100k.csv; keep rows with
    # status PENDING and total > 250.00; sum per
    # customer; return the top 10
start = time.perf_counter()
flat_query()
print(time.perf_counter() - start)
```

FIVE STEPS

1. Time the CSV version (left).
2. EXPLAIN ANALYZE the SQL query. Note the time.
3. CREATE INDEX idx_raw_orders_status_total ON raw_orders (status, order_total DESC);
4. Run EXPLAIN ANALYZE again.
5. Write two sentences comparing all three times.

PREDICT FIRST

Will the index make this query about 100 times faster? Write yes or no, and why.

BOSS LEVEL · WHAT WE MEASURED

The index helped, but not 100 times

Query (100,000 orders)	Time	Plan
CSV scan in Python	about 220 ms	every line
SQL, before the index	26.6 ms	Seq Scan
SQL, after the index	21.2 ms	Bitmap Index Scan

Why so small a gain? About **21,000 of the 100,000 rows** match the filter. When a query needs a fifth of the table, an index saves little. In the earlier lookup of one customer (18 rows) it saved a factor of 90.

Lesson: an index is not magic. It pays off when the query picks a small slice. Your own numbers will differ because your CSV differs.

STUCK? TOTALLY NORMAL

Common Core and Boss problems

What you see	Likely cause	Fix
could not open file "./data/orders_100k.csv"	CSV not visible in the container	Check the volume mapping in compose; use the path as the container sees it
duplicate key value violates unique constraint "raw_orders_pkey"	Data loaded twice	TRUNCATE raw_orders; then \copy once
relation "raw_orders" already exists	CREATE TABLE run twice	Skip it, or DROP TABLE raw_orders; (deletes its data)
Index changes nothing	Query matches many rows, or stale statistics	Run ANALYZE raw_orders; and read the plan
Timings jump around	First run is cold	Run three times; use the later ones

Still stuck after two minutes? Raise a hand or write the exact message on the board.

EXIT QUIZ · 5 MINUTES, ON YOUR OWN

Six questions, one letter each

1. Two programs rewrite the same CSV and orders vanish. This is a…

A rollback · B index miss · C race condition · D syntax error

2. CHECK (order_total >= 0.00) will…

A refuse any row with a negative total · B sort the orders · C make totals unique · D delete cheap orders

3. An index helps most when a query…

A needs every row · B picks a small slice of rows · C runs on a tiny table · D only inserts data

4. In ACID, Atomicity means…

A queries are fast · B data is encrypted · C one user at a time · D all steps succeed or none count

5. Write-ahead logging protects against…

A slow queries · B losing committed data in a crash · C duplicate keys · D negative totals

6. EXPLAIN ANALYZE in front of a query…

A runs it and shows the plan with timings · B deletes unused indexes · C converts a CSV to a table · D makes a backup

ANSWER KEY

Check your own paper

1. C. The result depends on which program writes last.

2. A. CHECK refuses rows that break its rule.

3. B. An index shines when it can skip most of the table.

4. D. Atomic: all steps count, or none do.

5. B. The log is written first, so confirmed data survives.

6. A. It runs the query and reports the plan and the time.

REFLECTION · TAKE THESE HOME

Can you answer these in your own words?

1. Why can a flat file not be trusted when many programs write to it?

2. Why did an index help a lot for one customer but little for the group-by query?

3. Where should a rule like “no negative totals” live, and why?

MUDDIEST POINT: on a slip of paper, write the one idea from today that is still unclear. We start next time with the most common one. **Homework:** finish Core and the Boss Level, and write two sentences comparing your three timings.

WELCOME TO THE DATA UNIVERSE

Thank you. Questions?

Today a flat file failed three ways, and a database fixed each one with rules, indexes and transactions.

Dr.-Ing. Chukwuka Victor Obionwu